Complemento Práctico: Pensando en Objetos

Del Paradigma Estructurado a la Ingeniería de Software

Los apuntes oficiales nos dan las definiciones técnicas, pero entender la Programación Orientada a Objetos (POO) requiere un cambio de mentalidad. Ya no somos escritores de guiones (código lineal), ahora somos arquitectos de sistemas. En este documento, traduciremos la teoría abstracta a la práctica dentro de NetBeans, enfocándonos en cómo evitar los errores comunes al crear nuestras primeras clases y cómo visualizar la memoria de la computadora cuando usamos la palabra clave `new`.

Contenido

Principios de Reutilización

Entendiendo el principio fundamental de la reutilización de código y cómo las clases padre comparten sus características con las clases hijas.

Estableciendo Jerarquías

La palabra clave en Java que establece relaciones de herencia y cómo interpretar semánticamente las jerarquías de clases.

Modelado de Relaciones

La representación visual de las relaciones entre clases utilizando la notación de diagramas adecuada.

Control de Acceso Avanzado

El control de acceso en la herencia y los errores comunes al restringir la visibilidad de los miembros.

Clase vs. Objeto: El Cortante y la Galleta

La Clase es el Cortante

La clase actúa como un molde o plantilla que define la estructura y el comportamiento de los objetos que crearemos. Piensa en un cortante de galletas: es de metal o plástico, define la forma y el tamaño, pero no se come porque no contiene datos reales. Solo existe un cortante, pero sirve para fabricar miles de galletas idénticas en forma.

Cuando escribes `public class Auto` en Java, estás fabricando el cortante. Defines qué atributos tendrá (color, marca, modelo) y qué comportamientos podrá realizar (acelerar, frenar), pero aún no existe ningún auto real en la memoria.

El Objeto es la Galleta

El objeto es la instancia concreta que ocupa memoria RAM real. Es la galleta de masa que tiene olor, sabor y decoración específica. Cada galleta puede tener valores diferentes en sus atributos: una de chocolate, otra de vainilla, pero todas mantienen la misma forma del cortante.

Si rompes una galleta (modificas un objeto), el cortante no sufre cambios y las demás galletas tampoco se ven afectadas. Cuando ejecutas `new Auto()`, estás cocinando una galleta específica con sus propios valores únicos.

La Clase: Definición

Plantilla abstracta que existe en el código fuente

El Objeto: Instancia

Entidad concreta que ocupa memoria durante la ejecución

Relación: Uno a Muchos

Una clase puede generar infinitos objetos diferentes

La Tiranía del Nombre de Archivo

Uno de los errores más frustrantes que enfrentan los principiantes ocurre al intentar cambiar el nombre de una clase. Java tiene una regla inquebrantable que debes respetar siempre: si tu clase se declara como `public class Automovil`, el archivo DEBE llamarse exactamente `Automovil.java`. No hay excepciones ni negociaciones.

1

El Error del Novato

Vas a la pestaña "Projects" en NetBeans, haces clic derecho en el archivo y seleccionas "Rename" para cambiar manualmente el nombre. Al hacer esto, tu código se llena inmediatamente de líneas rojas de error. ¿Por qué? Rompiste la regla de correspondencia entre el nombre de la clase y el nombre del archivo.

2

La Solución Profesional

Nunca cambies nombres escribiendo directamente. En su lugar, haz clic derecho en el nombre de la clase dentro del código o en el archivo del proyecto. Selecciona **Refactor > Rename** del menú contextual. Escribe el nuevo nombre deseado y NetBeans se encargará automáticamente de todo.

3

Magia del Refactoring

NetBeans cambia simultáneamente el nombre del archivo, el nombre de la clase en su declaración y todas las menciones a esa clase en el resto de tu proyecto. Esta es la manera profesional de mantener la consistencia y evitar errores de compilación.

Regla de Oro: La función Refactor es tu mejor amiga. Úsala siempre que necesites renombrar clases, métodos o variables. Mantiene tu código limpio y sincronizado automáticamente.

Estado y Comportamiento: Anatomía de un Objeto

En los PDFs oficiales leemos constantemente sobre "Atributos" y "Métodos", pero estos términos abstractos cobran vida cuando los traducimos a términos de simulación del mundo real. Cada objeto en programación tiene dos componentes fundamentales que debemos comprender profundamente.

Atributos = El Estado

Los atributos representan "cómo está" el objeto en un momento determinado. Son las características que describen el objeto usando sustantivos y adjetivos. Por ejemplo, considera un **Ventilador**:

- `velocidad = 3`
- `encendido = true`
- `color = "blanco"`
- `marca = "Liliana"`

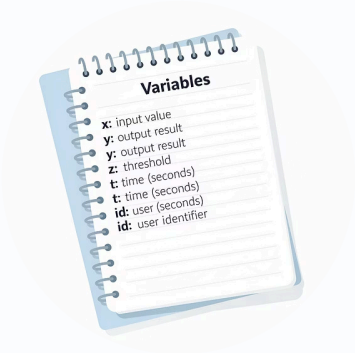
Si apagas el ventilador, su estado cambia (`encendido = false`), pero sigue siendo el mismo objeto en memoria. Los atributos son variables que almacenan el estado actual y pueden modificarse durante la vida del objeto.

Métodos = El Comportamiento

Los métodos representan "qué puede hacer" o "qué le podemos hacer" al objeto. Son las acciones expresadas con verbos. Los métodos son los responsables de modificar el estado interno del objeto.

- `encender()` - cambia `encendido` a `true`
- `apagar()` - cambia `encendido` a `false`
- `cambiarVelocidad(int nuevaVel)` - modifica el atributo `velocidad`
- `obtenerEstado()` - retorna información sobre el estado actual

Sin métodos, los objetos serían estructuras de datos estáticas e inútiles.



Estado Interno

Variables que definen las características actuales del objeto

Comportamiento Externo

Funciones que modifican el estado o realizan operaciones

Encapsulamiento

Los atributos privados solo se modifican mediante métodos públicos

La Magia de la Palabra `new`

La palabra clave `new` es probablemente la más importante de toda la unidad. Representa el momento exacto en que pasamos de la teoría a la práctica, del molde a la galleta, de la clase al objeto. Pero, ¿qué sucede realmente bajo el capó cuando escribimos esta palabra mágica?

Declaración

`Automovil miAuto;`

Aquí solo creamos una "etiqueta" o control remoto. Aún no hay auto real. La variable existe pero apunta a `null` (ninguna dirección de memoria).

Instanciación

`new Automovil();`

Java solicita al Sistema Operativo un bloque de memoria RAM libre. El constructor "dibuja" la estructura del auto en ese espacio (crea las variables color, marca, modelo).

Asignación

`miAuto = ...`

El operador `=` asigna la dirección de memoria del nuevo objeto a la variable `miAuto`. Ahora el control remoto está conectado al auto real.

```
// Paso 1: Declaración (solo la referencia)
Automovil miAuto;
// miAuto existe pero vale null

// Paso 2: Instanciación + Asignación
miAuto = new Automovil();
// Ahora miAuto apunta a un objeto real en memoria

// Forma compacta (todo en una línea)
Automovil otroAuto = new Automovil();
```

¿Qué ocurre en la RAM?

1. **Solicitud de Memoria:** `new` le pide al Sistema Operativo espacio libre en el heap (memoria dinámica)
2. **Construcción:** `Automovil()` ejecuta el constructor, inicializando todos los atributos del objeto
3. **Referencia:** Se retorna la dirección de memoria donde quedó almacenado el objeto
4. **Conexión:** La variable guarda esa dirección, permitiendo acceder al objeto mediante `miAuto.metodo()`

Sin el `new`, no hay objeto. Solo hay intención, una etiqueta sin nada que etiquetar.

Punto Clave: Puedes tener múltiples referencias apuntando al mismo objeto en memoria. Si modificas el objeto mediante una referencia, el cambio se verá en todas las demás porque todas apuntan al mismo espacio de memoria.

Ingeniería Inversa: Del Mundo Real al Código

Una de las habilidades más importantes que debes desarrollar como programador orientado a objetos es la capacidad de observar elementos del mundo real y traducirlos mentalmente a clases y objetos. Este proceso se llama **modelado** y requiere práctica constante. No se trata solo de copiar la realidad, sino de capturar su esencia relevante para nuestro sistema.

01

Identificar la Entidad

¿Qué cosa del mundo real quiero representar? Puede ser un objeto físico (Auto, Libro), una persona (Cliente, Profesor) o un concepto abstracto (Cuenta Bancaria, Pedido).

02

Extraer Atributos

¿Qué características definen a esta entidad? Busca sustantivos y adjetivos. Para un Libro: título, autor, ISBN, número de páginas, editorial, año de publicación.

03

Definir Métodos

¿Qué puede hacer esta entidad o qué podemos hacerle? Busca verbos. Para un Libro: prestar(), devolver(), consultarDisponibilidad(), actualizarEstado().

04

Aplicar Abstracción

¿Qué detalles son irrelevantes para mi sistema? Elimina todo lo que no necesites. Un Libro en una biblioteca no necesita atributos como "peso" o "color de portada".

Ejemplo Práctico: Teléfono Móvil

Atributos Posibles:

- marca, modelo, IMEI
- capacidadBateria, bateriaActual
- sistemaOperativo
- numeroTelefono
- estado (encendido/apagado)

Métodos Posibles:

- encender(), apagar()
- llamar(String numero)
- enviarMensaje(String numero, String texto)
- cargarBateria(int porcentaje)

La clave está en preguntarte constantemente: *"¿Para qué estoy construyendo este sistema?"* Un teléfono en un sistema de inventario de una tienda necesitará atributos diferentes que un teléfono en una aplicación de monitoreo de uso personal.

Abstracción Práctica: Ejercicio Mental

Para consolidar todo lo aprendido, practiquemos la **Abstracción**, que es la habilidad fundamental de ignorar los detalles irrelevantes y enfocarnos solo en lo esencial para nuestro sistema. La abstracción es lo que separa a un programador novato de uno experto.

Desafío: Modelar un Estudiante para el Sistema de la UTN

Imagina que te encargan desarrollar un sistema para gestionar estudiantes de la universidad. ¿Qué características deberías incluir en la clase `Estudiante`? Analicemos juntos:

❌ Color de Pelo

Irrelevante. No afecta su desempeño académico ni la gestión administrativa. Descartado.

✅ Legajo

Crítico. Identificador único del estudiante en el sistema. Es la clave primaria para buscar y organizar información.

❌ Sabe Cocinar

Irrelevante. No tiene relación con el propósito educativo del sistema. Descartado.

✅ Promedio

Esencial. Necesario para evaluar rendimiento, calcular becas y generar reportes académicos.

✅ Nombre y Apellido

Fundamental. Información básica de identificación necesaria en reportes y comunicaciones.

✅ Materias Cursadas

Importante. Permite validar correlatividades y generar el historial académico del estudiante.

```java
public class Estudiante {
    // Solo lo relevante (Abstracción aplicada)
    private int legajo;
    private String nombre;
    private String apellido;
    private double promedio;
    private ArrayList<Materia> materiasCursadas;

    // Constructor
    public Estudiante(int legajo, String nombre, String apellido) {
        this.legajo = legajo;
        this.nombre = nombre;
        this.apellido = apellido;
        this.promedio = 0.0;
        this.materiasCursadas = new ArrayList<>();
    }

    // Método para actualizar el promedio
    public void rendirExamen(Materia materia, int nota) {
        // Lógica para actualizar promedio general
        recalcularPromedio();
    }

    private void recalcularPromedio() {
        // Implementación del cálculo
    }
}
```

Conclusión Fundamental: Programar en objetos no es solo escribir código sintácticamente correcto. Es tomar decisiones conscientes sobre qué características del mundo real merecen estar en nuestro sistema y cuáles debemos ignorar. La abstracción es el arte de la simplificación inteligente.

Repaso Final: Checklist del Programador Orientado a Objetos

Hemos recorrido un camino desde las analogías visuales hasta el código concreto en NetBeans. Antes de cerrar, asegurémonos de que todos los conceptos fundamentales estén claros. Utiliza esta lista como referencia rápida cada vez que crees una nueva clase.

Clase = Molde

Define la estructura y comportamiento. Existe en el código fuente. Es reutilizable infinitas veces.

Objeto = Instancia

Ocupa memoria RAM. Tiene valores específicos. Se crea con `new`. Es único e independiente.

Refactor, No Rename

Siempre usa Refactor > Rename en NetBeans para mantener sincronizado el código y los archivos.

Estado

Atributos que describen cómo está el objeto

Comportamiento

Métodos que definen qué puede hacer

Identidad

Referencia única en memoria (dirección)

Preguntas Clave antes de Codificar

1. ¿Qué entidad del mundo real estoy modelando?
2. ¿Qué atributos son **realmente necesarios** para mi sistema?
3. ¿Qué comportamientos debe tener esta entidad?
4. ¿Estoy aplicando correctamente la abstracción?
5. ¿El nombre de mi archivo coincide con el nombre de la clase pública?
6. ¿Estoy usando `new` para crear instancias?

Errores Comunes a Evitar

- Cambiar nombres de archivo manualmente sin Refactor
- Declarar variables sin instanciar (`null` pointer)
- Incluir atributos irrelevantes (falta de abstracción)
- Confundir clase con objeto en las explicaciones
- No usar constructores apropiadamente
- Hacer públicos todos los atributos (rompe encapsulamiento)

Recuerda que el cambio de mentalidad del paradigma estructurado al orientado a objetos no sucede de la noche a la mañana. Requiere práctica constante, cometer errores y aprender de ellos. Cada clase que diseñes, cada objeto que instancies, te acercará más a pensar naturalmente como un arquitecto de software. ¡Sigue practicando y nos vemos en la próxima unidad!